Response paths and sustained recovery

Proposed UX and workflow scope, 21 October 2026. The interactive mockup uses only synthetic data and performs no service, approval, Git or infrastructure operation. Open `prototype/change-response/index.html` through the local prototype server.

Recommended decision model

Ask two questions: what restores service now, and what makes the fix durable? One incident can have a mitigation change and a separate permanent-fix change. Track their owners, evidence, approvals and outcomes independently. Keep them linked in the parent incident, external issue mapping and graphical workflow.

Operator choice	Immediate work	Durable work	Required decision
Mitigate now + fix below	One known reversible production mitigation	Lower Jarvis reproduces, fixes, tests and returns a Git change for promotion	Approve mitigation now; review permanent promotion later
Fix below, standard promotion	No immediate production mutation	Lower environment, CI/CD, exact tested artifact, upper approval	Accept lead time and separately approve promotion
Emergency runtime patch	One narrow scoped change with captured previous state	Mandatory Git reconciliation and permanent-fix task	Emergency approver, rationale, review deadline and rollback policy
Expedited Git hotfix	Reviewed immutable artifact directly to production	Back-merge and full pipeline verification across affected environments	Minimum required checks and an explicit waiver of omitted environment tests

Direct production source editing or an unrestricted `git pull` is not the proposed Git route. Git records the change; the deployment adapter applies a reviewed artifact. A faster route can shorten or waive specified checks under policy; it must not erase the change record, target binding, approval or recovery plan. The operator selects the route, with suggestions supported by impact, available rollback, change reversibility, known-good artifacts and environment readiness. A language model's preference is not authorization.

Four screens

1. **Choose response:** show observed impact, packet freshness, affected resources, known rollback and four choices. Recommend paired mitigation/permanent work for active impact when a safe reversible mitigation exists; explain missing prerequisites rather than invent a fix. 2. **Review change:** show exact target, original requester, patch/diff or artifact, source revision, generated scripts, scope, omitted checks, previous state and rollback conditions. Bind approval to their digests and current policy. 3. **Validate recovery:** show signal values, baseline, observation duration, freshness, stop conditions and outcomes. Let the operator see healthy, regression, missing telemetry and incomplete-window states. 4. **Permanent fix:** show parent/child tickets, Jarvis delegation, Git PR/MR, reproduction and CI receipts, immutable artifact, promotion gate and drift reconciliation. Service recovery does not imply this work is complete.

Watcher: start with a workflow branch

Add a reusable read-only verification activity after a recorded change, not a second incident engine. It consumes an exact change receipt, selected targets, original-user authorization and a versioned watch policy. A bounded scheduler can resume that durable workflow activity; it does not need continuous LLM analysis. Only introduce a separate engine if measured scale requires it.

The policy defines baseline window, signal queries, resource/revision identity, thresholds, required healthy duration, sample freshness, allowed gaps, maximum watch duration and regression/rollback conditions. The prototype's 15-minute baseline, ten-minute healthy window and twenty-minute maximum are illustrative, not universal defaults. Evaluate error rate, readiness and an application probe; alert disappearance alone does not establish recovery. Failures restart the healthy window. Missing, stale or insufficient samples yield **inconclusive**.

Proposed branches:

• **Healthy window complete:** record evidence-supported service recovery. Keep

temporary mitigation and permanent-change work open until reconciled.

• **Regression:** preserve observations and propose rollback. Execute only if the

exact approved plan includes that conditional rollback and current authority, target and revision still match; otherwise request approval.

• **Telemetry gap or deadline:** hold verification and escalate. Do not mark

recovered, blindly repeat a command, or treat silence as health.

• **Permission revoked / cancelled:** stop authorized reads/actions and preserve

the durable state for the owning operator. Do not silently substitute identity.

A mitigation review deadline triggers reassessment/escalation. It does not blindly undo a change keeping service alive. Changes with irreversible side effects or schema compatibility concerns require a suitable recovery strategy before an emergency path is offered. No generic rollback promise is made.

Acceptance for implementation

All four paths must retain source evidence and exact plan digests. Prove the normal route and each refusal: stale approval, changed artifact, wrong target, withdrawn grant, unavailable lower environment and unauthorized test waiver. A patch must create its reconciliation task before execution. A successful command with failing probes cannot close the incident. A missing-sample window cannot pass. A duplicate watcher wake cannot deploy or roll back twice. Healthy service with unresolved drift remains visible. Prove linked status synchronization without mapping an internal mitigation to an external resolved state prematurely.

The current mockup tests route selection, approval preview gating, four health states, lower-verified promotion and layout at 390/768/1440 pixels. These tests verify design behavior only. Production authority, watchers, Git operations and cross-swarm execution remain roadmap work.